

ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ИРКУТСКОЙ ОБЛАСТИ
«АНГАРСКИЙ ТЕХНИКУМ СТРОИТЕЛЬНЫХ ТЕХНОЛОГИЙ»

Курсовая работа

Тема: Разработка веб-сервиса "Интернет магазин компьютеров"

Разработал _____ Ошаров Ю.О

Руководитель _____ Денисюк А.В

Ангарск, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....	8
1.1 Общая характеристика разрабатываемого интернет-магазина компьютеров	8
1.2 Основы создания веб-сервисов.....	10
1.3 Редактор кода Zed в процессе разработки.....	12
1.4 Клиент-серверная архитектура веб-приложения	14
1.5 Контейнеризация и использование Docker.....	17
1.6 Фреймворк Express в серверной части проекта	19
1.7 Фреймворк Vue в клиентской части проекта	21
1.8 База данных.....	24
1.9 Хеширование паролей с использованием bcrypt	27
1.10 Взаимосвязь используемых технологий в архитектуре проекта.....	29
2.РАЗРАБОТКА СЕРВИСА.....	31
2.1.Архитектура системы	31
2.1.1 Архитектура Backend.....	31
2.1.2 Архитектура Frontend	31
2.1.3 Файлы docker	32
2.2 Модель данных проекта	32
2.2.1 Пользователь.....	32

						КР-09.02.07-О-19-26ПЗ					
Изм.	Кол.уч	Лист	№ док.	Подпись	Дата						
Разраб.		Ошаров Ю.О				Разработка web-сервиса "Интернет-магазин компьютеров"			<u>Лист</u>	Лист	Листов
Пров.		Денисюк А.В								3	42
Н.контр											
Утв.											
						ГАПОУ ИО АТСТ					

ВВЕДЕНИЕ

В современных условиях персональный компьютер остаётся одним из основных инструментов для учёбы, работы, программирования, обработки графики, игр и выполнения повседневных цифровых задач. При этом выбор готового компьютера или отдельных комплектующих часто вызывает затруднения у пользователя, так как рынок компьютерной техники включает большое количество моделей процессоров, видеокарт, материнских плат, оперативной памяти, накопителей, блоков питания и корпусов. Каждый компонент имеет собственные технические характеристики, а его выбор зависит не только от цены, но и от совместимости с другими элементами системы.

Интернет-магазины компьютерной техники позволяют упростить процесс выбора товара, однако обычного каталога с перечнем комплектующих часто недостаточно. Пользователь должен понимать, подходит ли выбранный процессор к материнской плате, достаточно ли мощности блока питания, соответствует ли оперативная память требованиям системы и не будет ли один компонент ограничивать производительность другого. Поэтому актуальной задачей является разработка веб-приложения, которое объединяет функции интернет-магазина и конфигуратора персонального компьютера.

Темой данной курсовой работы является разработка интернет-магазина компьютеров. В рамках проекта рассматривается веб-приложение CompCraft, предназначенное для просмотра компьютерных комплектующих, подбора конфигурации ПК, проверки совместимости компонентов, анализа возможного ограничения производительности, работы с личным профилем пользователя и выполнения административных операций. Такой подход позволяет рассматривать систему не только как витрину товаров, но и как прикладной сервис, помогающий пользователю сформировать технически корректную сборку.

						КР-09.02.07-О-19-26ПЗ	Лист
							5
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Актуальность темы обусловлена тем, что процесс самостоятельной сборки компьютера требует знаний о характеристиках комплектующих и правилах их совместимости. Ошибки при подборе могут привести к невозможности сборки, нестабильной работе системы или нерациональному расходованию бюджета. Разработка веб-приложения, которое предоставляет каталог компонентов и помогает пользователю проверять их совместимость, имеет практическую значимость, так как подобный сервис может использоваться как учебный проект, демонстрирующий работу современных веб-технологий, клиент-серверной архитектуры, базы данных и механизмов авторизации.

Целью курсовой работы является разработка веб-приложения интернет-магазина компьютеров с возможностью подбора и сохранения конфигураций персонального компьютера.

Для достижения поставленной цели необходимо решить следующие задачи:

- Изучить предметную область интернет-магазинов компьютерных комплектующих и определить основные требования к функциональности приложения.
- Рассмотреть основы создания веб-сервисов и клиент-серверной архитектуры.
- Обосновать выбор используемого технологического стека: Vue, Express, MySQL, Docker и bcrypt.
- Описать роль редактора кода Zed в процессе разработки проекта.
- Спроектировать структуру клиентской и серверной частей веб-приложения.
- Разработать базу данных для хранения пользователей, комплектующих и пользовательских конфигураций.
- Реализовать регистрацию, авторизацию и безопасное хранение паролей.

- Реализовать каталог комплектующих, конфигуратор сборки ПК, проверку совместимости и работу с профилем пользователя.
- Обеспечить возможность развёртывания проекта с использованием контейнеризации Docker.

Объектом исследования является процесс разработки веб-приложения для интернет-магазина компьютерных комплектующих.

Предметом исследования являются технологии и архитектурные решения, применяемые при создании интернет-магазина компьютеров: клиент-серверная архитектура, серверный фреймворк Express, клиентский фреймворк Vue, база данных MySQL, контейнеризация Docker и механизм хеширования паролей bcrypt.

Практическая значимость работы заключается в создании готового веб-приложения, которое может использоваться как учебный пример интернет-магазина компьютерных комплектующих. Проект демонстрирует работу с каталогом товаров, пользовательской авторизацией, сохранением данных, проверкой совместимости компонентов и административным управлением. Кроме того, приложение может быть доработано и расширено: например, в него можно добавить оформление заказов, онлайн-оплату, систему отзывов, избранное, сравнение комплектующих и рекомендации по подбору сборки.

1. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Общая характеристика разрабатываемого интернет-магазина компьютеров

Разрабатываемое веб-приложение представляет собой интернет-магазин компьютерных комплектующих с возможностью подбора конфигурации персонального компьютера. В отличие от обычного каталога товаров, где пользователь только просматривает список позиций, данный проект ориентирован на более сложный сценарий работы: пользователь может изучать комплектующие, подбирать совместимые элементы системы, сохранять собственные сборки и анализировать их характеристики. Поэтому теоретическая часть курсовой работы рассматривает только те технологии и подходы, которые непосредственно связаны с реализованной архитектурой приложения.

Основная предметная область проекта связана с онлайн-подбором комплектующих для персонального компьютера. В системе представлены такие категории товаров, как процессоры, видеокарты, материнские платы, оперативная память, накопители, блоки питания, корпуса и другие элементы, необходимые для сборки ПК. Для каждого компонента важны не только название и цена, но и технические характеристики: сокет процессора, тип памяти, форм-фактор, энергопотребление, объём, частота, производительность и другие параметры. Именно эти данные позволяют системе не просто показывать товары пользователю, а выполнять прикладную логику проверки совместимости.

Разрабатываемый интернет-магазин можно рассматривать как информационную систему, которая объединяет несколько функций. Во-первых, это каталог компьютерных комплектующих, где пользователь получает доступ к товарам и их характеристикам. Во-вторых, это конфигуратор сборки ПК, позволяющий выбрать набор компонентов для

						КР-09.02.07-О-19-26ПЗ	Лист
							8
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

будущего компьютера. В-третьих, это система проверки совместимости, которая помогает избежать ошибок при подборе комплектующих.

В-четвёртых, это пользовательский сервис, так как приложение поддерживает регистрацию, авторизацию, профиль пользователя и сохранение конфигураций. Также в проекте присутствует административная часть, необходимая для управления данными.

Особенность данной предметной области заключается в том, что компьютерные комплектующие имеют сложные связи между собой. Например, процессор должен соответствовать сокету материнской платы, оперативная память должна поддерживаться выбранной платой, корпус должен подходить по форм-фактору, а блок питания должен иметь достаточную мощность для всей сборки. Для пользователя без глубоких технических знаний самостоятельный подбор комплектующих может быть затруднительным. Поэтому веб-приложение решает не только задачу отображения товаров, но и задачу помощи в принятии технически корректного решения.

С точки зрения программной реализации проект построен как клиент-серверное веб-приложение. Клиентская часть отвечает за интерфейс, отображение страниц, работу с формами, карточками комплектующих, конфигуратором и профилем пользователя. Серверная часть обрабатывает запросы, выполняет бизнес-логику, обращается к базе данных, проверяет авторизацию и возвращает клиенту необходимые данные. Для хранения информации используется база данных MySQL, а для безопасной работы с паролями применяется хеширование с использованием bcrypt.

Таким образом, рассматриваемая система является не просто учебным сайтом, а полноценным веб-приложением, в котором объединены пользовательский интерфейс, серверная логика, база данных, авторизация, контейнеризация и средства разработки. Все последующие разделы

теоретической главы раскрывают именно те технологии, которые используются в готовом продукте.

1.2 Основы создания веб-сервисов

Создание веб-сервиса начинается с определения того, какие задачи должна выполнять система и каким образом пользователь будет взаимодействовать с ней через браузер. В рамках данного проекта веб-сервис предназначен для работы с каталогом компьютерных комплектующих, подбора конфигурации ПК, авторизации пользователей, сохранения пользовательских данных и выполнения административных операций. Поэтому архитектура приложения должна обеспечивать надёжный обмен данными между клиентской частью, сервером и базой данных.

Веб-сервис представляет собой программную систему, доступную по сети и обрабатывающую HTTP-запросы. Пользователь открывает приложение в браузере, после чего клиентская часть отправляет запросы на сервер. Сервер принимает запрос, определяет нужный маршрут, выполняет соответствующую логику и возвращает ответ. Ответ может содержать данные о комплектующих, информацию о пользователе, результаты проверки совместимости, сохранённые конфигурации или сообщение об ошибке.

Основой работы веб-сервиса является протокол HTTP. Он определяет, как клиент и сервер обмениваются данными. Например, при загрузке списка комплектующих клиент отправляет GET-запрос, так как ему необходимо получить данные. При регистрации пользователя или сохранении конфигурации используется POST-запрос, поскольку данные передаются на сервер для обработки и записи. При изменении информации может использоваться PUT- или PATCH-запрос, а при удалении — DELETE-запрос. Такое разделение методов делает структуру приложения более понятной и соответствует общепринятой логике веб-разработки.

						КР-09.02.07-О-19-26ПЗ	Лист
							10
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

В рассматриваемом проекте серверная часть построена на базе Express. Express позволяет создавать маршруты, которые соответствуют отдельным функциям приложения. Например, одни маршруты могут отвечать за получение списка товаров, другие — за авторизацию, третьи — за работу с пользовательскими сборками, четвёртые — за административные действия. Такой подход позволяет разделить функциональность по смысловым блокам и упростить сопровождение кода.

Важным элементом веб-сервиса является API. В данном случае API используется как посредник между Vue-интерфейсом и серверной логикой. Клиентская часть не обращается к базе данных напрямую. Она отправляет запрос на сервер, а сервер уже выполняет SQL-запросы к MySQL, проверяет права доступа, обрабатывает результат и отправляет клиенту подготовленные данные. Это повышает безопасность и позволяет централизованно контролировать бизнес-логику.

При создании веб-сервиса также необходимо учитывать обработку ошибок. Пользователь может отправить неправильные данные, запросить несуществующий товар, попытаться выполнить действие без авторизации или обратиться к ресурсу, который ему недоступен. Поэтому сервер должен возвращать понятные ответы с соответствующими HTTP-статусами. Например, ошибка авторизации может сопровождаться статусом 401, запрет доступа — 403, отсутствие ресурса — 404, а внутренняя ошибка сервера — 500. Такая система облегчает отладку и делает взаимодействие клиента с сервером предсказуемым.

Для интернет-магазина компьютеров особенно важна корректность данных. Если система показывает пользователю комплектующие, она должна брать информацию из единого источника — базы данных. Если пользователь сохраняет сборку, сервер должен убедиться, что выбранные компоненты существуют и могут быть связаны с конкретным пользователем. Если выполняется проверка совместимости, сервер должен использовать

						КР-09.02.07-О-19-26ПЗ	Лист
							11
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

технические характеристики товаров, а не случайные данные из интерфейса. Поэтому веб-сервис в проекте выполняет роль центрального слоя, который связывает пользовательские действия с данными и правилами предметной области.

Создание веб-сервиса также связано с вопросом развёртывания. Приложение должно запускаться не только на компьютере разработчика, но и в предсказуемой среде. Для этого в проекте используется Docker. Контейнеризация позволяет описать окружение приложения и базы данных так, чтобы запуск был одинаковым на разных устройствах. Это особенно важно для курсового проекта, так как преподаватель или другой пользователь может развернуть систему без ручной настройки всех зависимостей.

Таким образом, основа создания веб-сервисов в данном проекте включает несколько ключевых элементов: клиентскую часть, серверную часть, HTTP-взаимодействие, API, базу данных, обработку ошибок, авторизацию и контейнеризированное окружение. Все эти элементы работают совместно и обеспечивают функционирование интернет-магазина компьютеров как единой системы.

1.3 Редактор кода Zed в процессе разработки

Для разработки веб-приложения важен не только выбранный технологический стек, но и среда, в которой пишется и сопровождается код. В данном проекте в качестве редактора кода рассматривается Zed. Это современный редактор, ориентированный на высокую скорость работы, удобную навигацию по проекту и поддержку разработки с использованием современных инструментов.

Редактор кода играет важную роль при создании интернет-магазина компьютеров, поскольку проект состоит из нескольких частей: клиентского приложения на Vue, серверной части на Express, конфигурационных файлов, SQL-структуры базы данных, Docker-файлов и вспомогательных модулей. Разработчику необходимо быстро переключаться между компонентами,

						КР-09.02.07-О-19-26ПЗ	Лист
							12
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

находить нужные файлы, просматривать структуру проекта и исправлять ошибки. Zed подходит для такой работы, так как позволяет удобно работать с многофайловыми проектами.

Одним из преимуществ Zed является скорость открытия и редактирования файлов. Для веб-проекта это имеет практическое значение, так как разработчик часто работает одновременно с несколькими файлами: компонентом Vue, маршрутом Express, контроллером, моделью базы данных и файлом стилей. Если редактор быстро реагирует на действия пользователя, процесс разработки становится более удобным и менее утомительным.

В проекте интернет-магазина компьютеров структура кода разделена по назначению. Клиентская часть содержит компоненты интерфейса, страницы, маршрутизацию и работу с запросами. Серверная часть содержит маршруты, контроллеры, настройки подключения к базе данных и middleware. При такой организации редактор должен помогать ориентироваться в проекте. Zed позволяет работать с деревом файлов, искать по проекту, открывать несколько вкладок и быстро переходить к нужным участкам кода.

Для JavaScript- и Vue-разработки важна подсветка синтаксиса. Она позволяет визуально отличать переменные, функции, ключевые слова, строки, теги шаблонов и другие элементы кода. В проекте на Vue это особенно полезно, поскольку один компонент может содержать шаблон, логику и стили. Подсветка помогает быстрее находить ошибки и лучше понимать структуру компонента.

Также при разработке важна работа с терминалом. Веб-приложение обычно запускается через команды `npm` или через `Docker`. Разработчику необходимо устанавливать зависимости, запускать сервер, запускать клиентскую часть, выполнять миграции или импорт SQL-файла, просматривать ошибки в консоли. Возможность работать с терминалом рядом с кодом упрощает процесс разработки и тестирования.

Zed может использоваться не только для написания кода, но и для анализа структуры проекта. Например, при работе над серверной частью можно открыть файл маршрутов и связанный с ним контроллер, чтобы понять, как запрос от клиента доходит до конкретной функции. При работе над клиентской частью можно открыть Vue-компонент и файл, отвечающий за API-запросы. Такой подход помогает видеть связь между частями приложения.

В рамках учебного проекта редактор Zed выполняет прикладную роль: он используется как инструмент разработки и сопровождения кода. Он не влияет напрямую на архитектуру интернет-магазина, но помогает создавать и редактировать файлы, из которых состоит приложение. Поэтому его рассмотрение в теоретической части уместно именно как описание рабочей среды, применяемой при создании программного продукта.

Таким образом, Zed можно охарактеризовать как редактор, подходящий для разработки современного веб-приложения на Vue, Express и MySQL. Он помогает работать со структурой проекта, редактировать код, выполнять поиск, использовать терминал и поддерживать удобный процесс разработки. В контексте проекта CompCraft он выступает инструментом, с помощью которого создаётся и сопровождается исходный код интернет-магазина компьютеров.

1.4 Клиент-серверная архитектура веб-приложения

Клиент-серверная архитектура является основой рассматриваемого интернет-магазина компьютеров. Она предполагает разделение приложения на две основные части: клиентскую и серверную. Клиентская часть работает в браузере пользователя и отвечает за отображение интерфейса. Серверная часть работает отдельно и отвечает за обработку данных, обращение к базе, авторизацию и выполнение бизнес-логики.

В проекте клиентская часть реализована с использованием Vue. Она отвечает за страницы приложения, формы, карточки комплектующих,

элементы навигации, конфигуратор сборки и взаимодействие пользователя с системой. Пользователь видит именно клиентскую часть: каталог, кнопки, поля ввода, сообщения, таблицы и результаты проверки. Однако сама клиентская часть не хранит основные данные и не выполняет критически важные операции напрямую.

Серверная часть реализована на Node.js с использованием Express. Она принимает запросы от Vue-приложения, определяет нужный маршрут, обращается к базе данных MySQL и возвращает результат. Например, когда пользователь открывает каталог комплектующих, клиент отправляет запрос на сервер. Сервер получает этот запрос, извлекает данные из базы данных и отправляет их обратно клиенту. После этого Vue отображает полученные комплектующие на странице.

Такое разделение ответственности делает приложение более организованным. Клиент занимается интерфейсом, а сервер — данными и правилами работы системы. Если бы вся логика была сосредоточена только на клиенте, пользователь мог бы получить слишком большой доступ к внутренним данным. Если бы вся логика отображения находилась только на сервере, интерфейс был бы менее гибким. В проекте выбран современный подход, при котором Vue отвечает за динамический интерфейс, а Express предоставляет API.

Клиент-серверное взаимодействие происходит через HTTP-запросы. Каждый запрос имеет метод, адрес, заголовки и при необходимости тело запроса. Например, при входе в систему пользователь вводит email и пароль. Vue отправляет эти данные на сервер. Сервер проверяет пользователя в базе данных, сравнивает пароль с хешем и при успешной проверке формирует токен авторизации. После этого клиент получает информацию о результате входа.

Особое значение в клиент-серверной архитектуре имеет безопасность. Клиентская часть доступна пользователю, поэтому ей нельзя доверять

						КР-09.02.07-О-19-26ПЗ	Лист
							15
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

полностью. Пользователь может изменить данные в браузере, отправить запрос вручную или попытаться обратиться к закрытым маршрутам. По этой причине все важные проверки должны выполняться на сервере. В проекте это относится к авторизации, проверке роли пользователя, сохранению сборок, получению личных данных и административным операциям.

База данных MySQL является отдельным слоем системы. Клиент не имеет прямого доступа к базе. Все SQL-запросы выполняются только сервером. Такой подход защищает данные и позволяет контролировать, какие операции разрешены пользователю. Например, обычный пользователь может просматривать комплектующие и сохранять свои сборки, но не должен иметь возможность изменять данные всех товаров. Администратор, наоборот, может управлять содержимым каталога.

В проекте также реализована работа с состоянием пользователя. После входа в систему пользователь получает возможность работать с личным профилем и сохранёнными конфигурациями. Для этого сервер должен понимать, кто именно отправляет запрос. Обычно это решается с помощью токена, который передаётся вместе с запросами. Сервер проверяет токен и определяет пользователя. Такой механизм позволяет реализовать авторизацию без постоянного ввода логина и пароля.

Клиент-серверная архитектура удобна для расширения проекта. Если в дальнейшем потребуется добавить новые функции, например оформление заказа, сравнение комплектующих, избранное или рекомендации, это можно сделать через добавление новых серверных маршрутов и клиентских страниц. При этом основная структура приложения сохранится.

Таким образом, клиент-серверная архитектура в проекте CompCraft обеспечивает разделение интерфейса, бизнес-логики и данных. Vue реализует пользовательскую часть, Express обрабатывает запросы, MySQL хранит информацию, а обмен между компонентами осуществляется через HTTP API. Такая архитектура соответствует задачам интернет-магазина

						КР-09.02.07-О-19-26ПЗ	Лист
							16
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

компьютеров и позволяет поддерживать сложную логику подбора комплектующих.

1.5 Контейнеризация и использование Docker

Docker используется в проекте для контейнеризации приложения. Контейнеризация — это подход, при котором приложение и его зависимости упаковываются в изолированную среду. Благодаря этому программа может запускаться одинаково на разных компьютерах, независимо от особенностей локальной системы. Для веб-приложения, состоящего из клиентской части, серверной части и базы данных, такой подход особенно полезен.

В обычной разработке запуск проекта может требовать установки Node.js, MySQL, настройки переменных окружения, создания базы данных, установки зависимостей и выполнения дополнительных команд. Если это делать вручную, легко столкнуться с ошибками: разные версии Node.js, занятые порты, отсутствующие пакеты, неправильные параметры подключения к базе данных. Docker помогает избежать части этих проблем, так как окружение описывается заранее в конфигурационных файлах.

В проекте интернет-магазина компьютеров Docker нужен для развёртывания нескольких связанных сервисов. Обычно такая структура включает контейнер серверного приложения и контейнер базы данных MySQL. Серверный контейнер содержит Node.js-приложение, зависимости и команды запуска. Контейнер базы данных содержит MySQL и хранит данные. Эти контейнеры могут взаимодействовать между собой внутри общей Docker-сети.

Практическое значение Docker в данном проекте состоит в том, что приложение можно запустить без ручной установки всей инфраструктуры. Достаточно использовать подготовленные файлы конфигурации. Это удобно как для разработчика, так и для проверки курсового проекта. Если проект переносится на другой компьютер, не нужно заново подробно настраивать окружение: Docker создаёт одинаковые условия запуска.

						КР-09.02.07-О-19-26ПЗ	Лист
							17
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Одним из ключевых файлов контейнеризации является Dockerfile. В нём описывается, на основе какого образа создаётся контейнер, какие файлы копируются, какие зависимости устанавливаются и какая команда запускается. Например, для Node.js-приложения Dockerfile может использовать официальный образ Node, копировать package.json, устанавливать зависимости через npm и запускать сервер. Такой файл превращает процесс подготовки окружения в повторяемую инструкцию.

Для запуска нескольких контейнеров применяется docker-compose. Он позволяет описать сразу несколько сервисов: сервер, базу данных и при необходимости клиентскую часть. В docker-compose указываются порты, переменные окружения, тома для хранения данных и зависимости между сервисами. Например, сервер может зависеть от MySQL, так как без базы данных он не сможет полноценно обрабатывать запросы.

Для MySQL особенно важно использование постоянного хранилища данных. Если контейнер базы данных будет удалён, данные могут потеряться. Поэтому Docker позволяет подключать volume — специальное хранилище, которое сохраняет данные вне жизненного цикла контейнера. В учебном проекте это полезно, потому что база данных должна сохраняться между перезапусками.

Контейнеризация также упрощает командную работу. Если один разработчик настроил проект в Docker, другой может получить такой же результат, используя те же файлы. Это снижает зависимость от конкретной операционной системы. Проект можно запускать на Windows, Linux или macOS при наличии Docker.

В контексте архитектуры интернет-магазина Docker не заменяет Express, Vue или MySQL. Он выполняет инфраструктурную роль. Express отвечает за серверную логику, Vue — за интерфейс, MySQL — за данные, а Docker — за стабильное окружение, в котором всё это запускается. Поэтому

						КР-09.02.07-О-19-26ПЗ	Лист
							18
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

его использование связано не с бизнес-логикой магазина, а с развёртыванием и эксплуатацией приложения.

Таким образом, Docker в проекте CompCraft обеспечивает переносимость, удобство запуска и изоляцию окружения. Контейнеризация позволяет описать приложение как набор сервисов и уменьшить количество ошибок, связанных с ручной настройкой. Для курсового проекта это особенно важно, так как готовый продукт должен быть воспроизводимым и понятным при проверке.

1.6 Фреймворк Express в серверной части проекта

Express является основным фреймворком серверной части проекта. Он используется поверх Node.js и предоставляет удобные средства для создания веб-сервера, маршрутов и middleware. Для интернет-магазина компьютеров Express выполняет роль центрального слоя, через который проходят запросы от клиентского приложения к базе данных и бизнес-логике.

Node.js сам по себе позволяет создавать сервер, но без фреймворка разработка быстро становится сложной. Необходимо вручную обрабатывать URL, методы запросов, заголовки, ошибки и данные. Express упрощает эту работу. Он позволяет описывать маршруты в понятной форме, подключать промежуточные обработчики и разделять код по модулям. Это делает серверную часть более читаемой.

В проекте Express используется для построения API. Каждый маршрут соответствует определённому действию. Например, один маршрут может возвращать список комплектующих, другой — данные конкретного компонента, третий — выполнять вход пользователя, четвёртый — сохранять сборку. Благодаря маршрутам приложение получает чёткую структуру, где каждый URL связан с определённой функцией.

Особое значение имеет middleware. Middleware — это промежуточные функции, которые выполняются между получением запроса и отправкой ответа. В интернет-магазине компьютеров такие функции могут

						КР-09.02.07-О-19-26ПЗ	Лист
							19
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

использоваться для обработки JSON-данных, проверки токена авторизации, ограничения доступа, обработки ошибок и настройки CORS. Middleware позволяет вынести общую логику из контроллеров и не повторять её в каждом маршруте.

Например, если пользователь обращается к личному профилю, сервер должен проверить, авторизован ли он. Вместо того чтобы писать проверку внутри каждого обработчика, можно создать отдельный middleware авторизации. Он проверяет токен, получает данные пользователя и передаёт управление дальше. Если токен отсутствует или неверен, middleware возвращает ошибку. Такой подход делает код более чистым.

Важной частью Express-приложения являются контроллеры. Маршрут определяет, какой URL обрабатывается, а контроллер выполняет основную логику. Например, контроллер каталога получает данные из базы, контроллер авторизации проверяет пароль и создаёт токен, контроллер профиля обновляет данные пользователя. Разделение маршрутов и контроллеров помогает избежать слишком больших файлов и упрощает сопровождение проекта.

Express также удобен для работы с ошибками. В приложении могут возникать разные ситуации: товар не найден, пользователь не авторизован, данные формы некорректны, база данных недоступна. Express позволяет подключить общий обработчик ошибок, который будет формировать единый ответ для клиента. Это важно для Vue-приложения, так как клиентская часть должна понимать, что именно произошло, и показать пользователю корректное сообщение.

Серверная часть интернет-магазина также взаимодействует с MySQL. Express не работает с базой данных напрямую как СУБД, но он организует маршрут, внутри которого выполняется запрос к базе. Например, при получении списка товаров контроллер отправляет SQL-запрос через библиотеку подключения к MySQL, получает результат и возвращает его

						КР-09.02.07-О-19-26ПЗ	Лист
							20
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

клиенту в формате JSON. Таким образом, Express связывает HTTP-уровень и слой данных.

При работе с авторизацией Express также выполняет важную роль. Когда пользователь отправляет email и пароль, сервер принимает эти данные, ищет пользователя в MySQL, сравнивает пароль с помощью bcrypt и создаёт токен. После этого клиент может использовать токен для дальнейших запросов. Все эти действия происходят на серверной стороне, что защищает систему от прямого доступа к базе данных.

Для административной части Express позволяет ограничить маршруты по роли пользователя. Например, обычный пользователь может просматривать каталог, но не должен изменять список комплектующих. Административные маршруты должны быть доступны только пользователю с соответствующей ролью. Такая проверка реализуется на сервере, потому что клиентский интерфейс можно изменить вручную, а серверная проверка остаётся обязательной.

Таким образом, Express в проекте CompCraft является основой серверной архитектуры. Он обеспечивает маршрутизацию, middleware, обработку запросов, взаимодействие с базой данных, авторизацию, обработку ошибок и разделение логики по модулям. Использование Express оправдано тем, что он достаточно лёгкий, гибкий и хорошо подходит для создания API интернет-магазина компьютеров.

1.7 Фреймворк Vue в клиентской части проекта

Vue используется в проекте для реализации клиентской части веб-приложения. Это фреймворк JavaScript, предназначенный для создания интерактивных пользовательских интерфейсов. В интернет-магазине компьютеров Vue отвечает за отображение страниц, работу с компонентами, взаимодействие с пользователем и отправку запросов к серверу.

Главная особенность Vue заключается в компонентном подходе. Интерфейс приложения можно разделить на отдельные компоненты: шапку

						КР-09.02.07-О-19-26ПЗ	Лист
							21
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

сайта, карточку комплектующего, форму входа, страницу каталога, блок configurатора, профиль пользователя, административные элементы. Каждый компонент отвечает за свою часть интерфейса и может переиспользоваться в разных местах. Это делает код более организованным.

Для интернет-магазина компьютеров компонентный подход особенно полезен. Каталог комплектующих состоит из множества похожих карточек. Вместо того чтобы вручную описывать каждую карточку, можно создать один компонент и передавать в него данные товара. Если нужно изменить внешний вид карточки, достаточно изменить один компонент, а не каждую страницу отдельно.

Vue также удобен для работы с реактивными данными. Это означает, что при изменении данных интерфейс автоматически обновляется. Например, если пользователь выбирает процессор в configurаторе, состояние сборки изменяется, и интерфейс может сразу показать выбранный компонент, обновить итоговую стоимость или выполнить проверку совместимости. Такой подход делает приложение более динамичным и удобным.

В проекте Vue взаимодействует с сервером через API-запросы. Когда пользователь открывает страницу каталога, клиентская часть отправляет запрос к Express-серверу. Сервер возвращает данные из MySQL, а Vue отображает их в интерфейсе. Когда пользователь входит в систему, Vue отправляет email и пароль на сервер. Когда пользователь сохраняет конфигурацию, Vue передаёт выбранные комплектующие на серверную часть. Таким образом, Vue не заменяет сервер, а работает вместе с ним.

Маршрутизация на клиентской стороне позволяет создавать несколько страниц внутри одного приложения. Пользователь может переходить между каталогом, configurатором, страницей входа, профилем и административной панелью. При этом Vue управляет отображением нужных компонентов. Это делает приложение ближе к одностраничной архитектуре,

						КР-09.02.07-О-19-26ПЗ	Лист
							22
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

где переходы между разделами происходят быстрее и удобнее для пользователя.

Для интернет-магазина важна визуальная и функциональная ясность интерфейса. Пользователь должен быстро понимать, где находится каталог, как выбрать комплектующие, как сохранить сборку и где посмотреть профиль. Vue помогает строить интерфейс из логических блоков. Например, форма авторизации может быть отдельным компонентом, блок характеристик товара — отдельным компонентом, список сохранённых сборок — отдельным компонентом.

Vue также облегчает обработку пользовательского ввода. Формы регистрации, входа, редактирования профиля и добавления данных могут быть связаны с переменными состояния. Пользователь вводит данные, Vue сохраняет их в состоянии компонента, а затем отправляет на сервер. При ошибке сервер возвращает сообщение, которое Vue может отобразить рядом с формой. Это улучшает взаимодействие пользователя с системой.

В конфигураторе ПК Vue играет особенно важную роль. Подбор комплектующих — это интерактивный процесс. Пользователь последовательно выбирает компоненты, а система должна отображать текущий состав сборки. Если реализована проверка совместимости или анализ bottleneck, результат должен быть понятен пользователю. Vue позволяет обновлять такие данные без полной перезагрузки страницы, что делает работу с конфигуратором удобнее.

Несмотря на то что Vue отвечает за клиентскую часть, критическая логика не должна находиться только на стороне клиента. Например, проверка прав доступа, работа с паролями и изменение данных в базе должны выполняться на сервере. Vue может скрыть кнопку администратора от обычного пользователя, но окончательное ограничение доступа должен выполнять Express. Такое разделение повышает безопасность.

						КР-09.02.07-О-19-26ПЗ	Лист
							23
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Таким образом, Vue в проекте CompCraft используется как основа пользовательского интерфейса. Он обеспечивает компонентную структуру, реактивное обновление данных, работу со страницами, формами и API-запросами. Благодаря Vue приложение становится удобным для пользователя и поддерживаемым для разработчика.

1.8 База данных

MySQL используется в проекте как система управления реляционной базой данных. Она отвечает за хранение основной информации приложения: пользователей, комплектующих, характеристик, сохранённых сборок и других данных, необходимых для работы интернет-магазина компьютеров. Выбор реляционной базы данных оправдан тем, что данные проекта имеют чёткую структуру и связи между сущностями.

В интернет-магазине компьютеров данные нельзя хранить только в виде отдельных текстовых файлов или массивов в коде. Каталог комплектующих должен быть постоянным, пользователи должны сохраняться между перезапусками приложения, а конфигурации должны быть связаны с конкретными аккаунтами. MySQL решает эти задачи, предоставляя надёжное хранилище с поддержкой таблиц, связей и SQL-запросов.

Основной объект предметной области — компьютерный компонент. Компонент может иметь название, категорию, цену, описание, изображение и набор технических характеристик. В отличие от простого товара, компьютерная комплектующая требует более подробного описания. Например, для процессора важны сокет, количество ядер, частота и энергопотребление; для видеокарты — объём видеопамяти и производительность; для материнской платы — чипсет, сокет, тип памяти и форм-фактор. Поэтому структура базы данных должна учитывать разнообразие характеристик.

						КР-09.02.07-О-19-26ПЗ	Лист
							24
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

В проекте также важна сущность пользователя. Пользователь может зарегистрироваться, войти в систему, редактировать профиль и сохранять сборки. Для этого в базе данных должна быть таблица пользователей. В ней хранятся идентификатор, имя, email, хеш пароля и дополнительные данные. Пароль не должен сохраняться в открытом виде, поэтому в таблице хранится только результат хеширования.

Связь между пользователем и сохранённой сборкой является важной частью проекта. Одна учётная запись может иметь несколько конфигураций. Каждая конфигурация может включать несколько комплектующих. Это типичная реляционная связь: пользователь связан со сборками, а сборки связаны с компонентами. MySQL хорошо подходит для такой модели, так как позволяет использовать внешние ключи и соединения таблиц.

Для конфигуратора ПК база данных играет не только роль хранилища, но и источника данных для проверки совместимости. Если в таблицах хранятся технические характеристики комплектующих, сервер может сравнивать их между собой. Например, сокет процессора должен совпадать с сокетом материнской платы. Тип оперативной памяти должен поддерживаться платой. Мощность блока питания должна быть достаточной для выбранных компонентов. Такие проверки невозможны без структурированных данных.

SQL-запросы используются для получения, добавления, изменения и удаления информации. При открытии каталога выполняется запрос на получение списка комплектующих. При просмотре подробной информации о товаре выполняется запрос по идентификатору. При регистрации пользователя выполняется INSERT-запрос. При редактировании профиля выполняется UPDATE-запрос. При удалении записи в административной панели может использоваться DELETE-запрос. Таким образом, вся прикладная логика, связанная с постоянными данными, опирается на MySQL.

						КР-09.02.07-О-19-26ПЗ	Лист
							25
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

MySQL также позволяет выполнять фильтрацию и сортировку. Для интернет-магазина компьютеров это важно, потому что пользователь может искать комплектующие по категории, цене, названию или характеристикам. Например, можно получить только видеокарты, только процессоры определённого сокета или комплектующие в заданном ценовом диапазоне. Такие запросы повышают удобство работы с каталогом.

Для административной части база данных также является центральным элементом. Администратор должен иметь возможность управлять данными каталога: добавлять комплектующие, изменять характеристики, редактировать цены и удалять устаревшие позиции. Все эти изменения должны сохраняться в MySQL, чтобы пользовательская часть отображала актуальную информацию.

Реляционная модель также помогает поддерживать целостность данных. Например, сохранённая сборка не должна ссылаться на несуществующего пользователя. Запись о составе сборки не должна ссылаться на несуществующий компонент. Для этого применяются связи между таблицами. Такая организация делает данные более надёжными и уменьшает вероятность логических ошибок.

Важным преимуществом MySQL является распространённость и понятность SQL. Для курсового проекта это имеет значение, так как структура базы данных может быть легко показана, объяснена и проверена. SQL-запросы позволяют наглядно продемонстрировать, как приложение работает с данными. Кроме того, MySQL хорошо сочетается с Node.js и Express через специальные библиотеки подключения.

Таким образом, MySQL в проекте CompCraft используется как основное средство хранения данных. Она обеспечивает постоянное хранение пользователей, комплектующих, характеристик и сборок, поддерживает связи между сущностями, позволяет выполнять фильтрацию и служит

основой для проверки совместимости. Без базы данных проект не мог бы работать как полноценный интернет-магазин компьютеров.

1.9 Хеширование паролей с использованием bcrypt

Безопасное хранение паролей является обязательной частью любого веб-приложения, где есть регистрация и авторизация пользователей. В проекте интернет-магазина компьютеров пользователь создаёт учётную запись, входит в систему и получает доступ к личному профилю и сохранённым сборкам. Поэтому сервер должен защищать пароль пользователя и не хранить его в открытом виде.

Для решения этой задачи используется bcrypt. Это библиотека, предназначенная для хеширования паролей. Хеширование отличается от шифрования тем, что оно является необратимым процессом. Если пароль был превращён в хеш, восстановить исходный пароль из хеша нельзя. При авторизации система не расшифровывает пароль, а сравнивает введённое значение с сохранённым хешем.

При регистрации пользователь вводит пароль. Сервер получает его, но перед записью в базу данных передаёт в bcrypt. Библиотека создаёт хеш, который и сохраняется в таблице пользователей. Исходный пароль в базу данных не попадает. Это важно, потому что даже при несанкционированном доступе к базе злоумышленник не увидит реальные пароли пользователей.

bcrypt использует соль. Соль — это дополнительное случайное значение, которое применяется при хешировании. Благодаря соли два одинаковых пароля у разных пользователей не будут иметь одинаковые хеши. Это усложняет массовый подбор паролей и делает защиту надёжнее. В отличие от простых хеш-функций, bcrypt специально рассчитан на хранение паролей и замедляет перебор.

В проекте bcrypt применяется в двух основных сценариях. Первый сценарий — регистрация пользователя. Сервер получает пароль, хеширует его и сохраняет результат. Второй сценарий — вход в систему. Пользователь

вводит email и пароль, сервер находит пользователя в базе данных и сравнивает введенный пароль с сохранённым хешем. Если сравнение успешно, пользователь считается подлинным.

Использование bcrypt также важно при смене пароля. Если пользователь в профиле меняет пароль, сервер должен сначала проверить текущий пароль. Для этого введенный текущий пароль сравнивается с существующим хешем. Затем новый пароль снова хешируется и записывается в базу. Это обеспечивает единый подход к безопасности во всех сценариях.

Хранение паролей в открытом виде является недопустимым. Если база данных будет скомпрометирована, все аккаунты пользователей окажутся под угрозой. Кроме того, многие пользователи применяют один и тот же пароль на разных сайтах. Поэтому утечка паролей из одного учебного проекта может привести к проблемам и в других сервисах. Даже в курсовой работе важно показывать правильный подход к защите данных.

bcrypt не отменяет другие меры безопасности. Помимо хеширования, необходимо проверять входные данные, ограничивать доступ к закрытым маршрутам и корректно работать с токенами авторизации. Однако именно bcrypt отвечает за то, чтобы пароль не хранился в базе в исходном виде. Это один из ключевых элементов безопасности проекта.

В связке с Express bcrypt используется на серверной стороне. Клиентская часть Vue не должна хешировать пароль вместо сервера, потому что клиентский код доступен пользователю и не является надёжной средой. Пользователь отправляет пароль по запросу, а сервер уже выполняет безопасную обработку. После проверки пароля сервер может выдать токен авторизации, который будет использоваться для дальнейшей работы с приложением.

Таким образом, bcrypt в проекте CompCraft применяется для безопасного хранения и проверки паролей пользователей. Он защищает

						КР-09.02.07-О-19-26ПЗ	Лист
							28
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

учётные записи, снижает риски при возможной утечке базы данных и соответствует требованиям к разработке веб-приложений с авторизацией.

1.10 Взаимосвязь используемых технологий в архитектуре проекта

Все рассмотренные технологии в проекте не существуют отдельно, а образуют единую архитектуру веб-приложения. Vue отвечает за клиентский интерфейс, Express — за серверную обработку запросов, MySQL — за хранение данных, bcrypt — за безопасность паролей, Docker — за развёртывание, а Zed — за разработку и сопровождение кода. Каждая технология выполняет свою роль и связана с конкретными функциями интернет-магазина компьютеров.

Пользователь начинает работу с клиентской части. Он открывает приложение в браузере и видит интерфейс, созданный на Vue. Через этот интерфейс он может просматривать каталог комплектующих, выбирать категории, работать с конфигуратором, регистрироваться, входить в систему и открывать профиль. Все действия пользователя преобразуются в запросы к серверу.

Серверная часть на Express принимает эти запросы. Если пользователь запрашивает каталог, сервер обращается к MySQL и получает список комплектующих. Если пользователь входит в систему, сервер получает email и пароль, ищет пользователя в базе и проверяет пароль через bcrypt. Если пользователь сохраняет сборку, сервер связывает её с учётной записью. Если запрос относится к административной функции, сервер проверяет права доступа.

MySQL хранит данные, необходимые для работы всех разделов приложения. Без базы данных невозможно было бы получить список комплектующих, сохранить пользователя, отобразить профиль или восстановить ранее созданную конфигурацию. Поэтому база данных является основой постоянного состояния системы.

						КР-09.02.07-О-19-26ПЗ	Лист
							29
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Docker обеспечивает запуск всей системы в едином окружении. Он помогает связать серверное приложение и базу данных, снижает зависимость от локальных настроек и делает проект более удобным для развёртывания. Это важно для курсовой работы, так как готовое приложение должно быть не только написано, но и воспроизводимо запущено.

Zed используется как рабочий инструмент разработчика. В нём создаются и редактируются файлы проекта: Vue-компоненты, серверные маршруты, контроллеры, SQL-скрипты и Docker-конфигурации. Хотя редактор не является частью исполняемой архитектуры, он влияет на процесс разработки и удобство сопровождения проекта.

На уровне логики проект можно представить как цепочку: пользовательское действие — Vue-компонент — HTTP-запрос — маршрут Express — контроллер — база данных MySQL — ответ сервера — обновление интерфейса. Если действие связано с авторизацией, в эту цепочку добавляется bcrypt и механизм токенов. Если действие выполняется в развёрнутой среде, все элементы работают внутри Docker-контейнеров или через Docker-инфраструктуру.

Такое распределение технологий позволяет избежать смешения ответственности. Интерфейс не хранит пароли и не работает напрямую с базой. Сервер не занимается визуальным отображением страниц. База данных не выполняет пользовательскую логику, а только хранит и выдаёт данные. Docker не изменяет код приложения, а обеспечивает среду запуска. Благодаря этому проект становится более структурированным.

Для интернет-магазина компьютеров такая архитектура является оправданной. Каталог комплектующих требует работы с данными, конфигуратор требует интерактивного интерфейса, авторизация требует серверной безопасности, а развёртывание требует стабильной среды. Используемые технологии соответствуют этим задачам и не являются случайными.

2.РАЗРАБОТКА СЕРВИСА

2.1.Архитектура системы

2.1.1 Архитектура Backend

Серверная часть размещена в папке backend и разработана на платформе Node.js с использованием фреймворка Express. Она отвечает за обработку запросов, реализацию бизнес-логики, взаимодействие с базой данных и обеспечение безопасности.

Основные элементы серверной части:

- app.js — точка входа в приложение, где выполняется настройка сервера, подключение middleware и маршрутов;
- config — конфигурационные файлы для подключения к базе данных и настройки сессий;
- controllers — контроллеры, реализующие обработку запросов пользователей;
- models — модели данных и логика взаимодействия с таблицами базы данных;
- routes — маршруты API;
- middleware — промежуточные обработчики запросов;
- services — сервисный слой с бизнес-логикой конфигулятора;
- scripts/seed.js — скрипт для заполнения базы тестовыми данными.

В приложении А предоставлена структура архитектуры backend

2.1.2 Архитектура Frontend

Клиентская часть размещена в папке frontend и реализована с использованием Vue 3 и сборщика Vite.

Основные элементы клиентской части:

- main.js — точка входа клиентского приложения;
- App.vue — корневой компонент приложения;

						КР-09.02.07-О-19-26ПЗ	Лист
							31
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

- src — каталог компонентов, страниц, сервисов и вспомогательных модулей;
- style.css — глобальные стили интерфейса.

В приложении Б предоставлена структура архитектуры frontend

2.1.3 Файлы docker

Для разработки и тестирования с помощью docker в проекте существуют файлы:

- Dockerfile — описание контейнеризации приложения;
- docker-compose.yml — конфигурация совместного запуска сервисов;
- docker/nginx.conf — настройки веб-сервера Nginx;
- README.md — инструкция по установке и запуску проекта.

2.2 Модель данных проекта

Для хранения данных используется реляционная база данных MySQL.

Основные сущности системы:

- пользователь;
- комплектующее;
- конфигурация;
- элементы конфигурации;
- рекомендации.

2.2.1 Пользователь

Хранит регистрационные данные и информацию о правах доступа. Для каждого пользователя сохраняются:

- имя;
- адрес электронной почты;
- хеш пароля;
- роль в системе.

						КР-09.02.07-О-19-26ПЗ	Лист
							32
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

2.2.2 Комплектующее

Представляет отдельный компонент персонального компьютера. В базе хранятся:

- название;
- категория компонента;
- технические характеристики;
- стоимость;
- производитель;
- наличие.

2.2.3 Конфигурация

Содержит сведения о пользовательской сборке:
название конфигурации;

- владелец;
- дата создания;
- общая стоимость;
- назначение сборки.

2.2.4 Рекомендации

Используются для автоматического подбора и улучшения сборки на основе выбранных компонентов.

в приложении В предоставленна ER-диаграмма базы данных

2.3 Реализация системы аутентификации и авторизации

В проекте реализована полноценная система регистрации и входа пользователей.

При регистрации выполняется:

- проверка корректности введенных данных;
- проверка уникальности адреса электронной почты;
- хеширование пароля;
- сохранение пользователя в базе данных.

На рисунке 1 предоставлена функция register.

```
export async function register(req, res) {
  const { username, email, password } = req.body;
  if (await User.findByEmail(email)) return res.status(409).json({ error: { code: 'EMAIL_EXISTS', message: 'Email уже зарегистрирован' } });
  const passwordHash = await bcrypt.hash(password, Number(process.env.BCRYPT_ROUNDS || 12));
  const user = await User.create({ username, email, passwordHash });
  req.session.userId = user.id;
  await logActivity({ ...req, user }, 'auth.register');
  res.status(201).json({ user, accessToken: signAccessToken(user), refreshToken: signRefreshToken(user) });
}
```

Рис. 1 функция register

При авторизации система:

- проверяет существование пользователя;
- сравнивает введенный пароль с сохранённым хешем;
- создаёт пользовательскую сессию;
- предоставляет доступ к защищённым разделам системы.

На рисунке 2 предоставлена функция login.

```
export async function login(req, res) {
  const { email, password } = req.body;
  const user = await User.findByEmail(email);
  if (!user || !(await bcrypt.compare(password, user.password_hash))) return res.status(401).json({ error: { code: 'INVALID_CREDENTIALS', message: 'Неверный email или пароль' } });
  if (user.is_banned) return res.status(403).json({ error: { code: 'BANNED', message: 'Пользователь заблокирован' } });
  req.session.userId = user.id;
  await logActivity({ ...req, user }, 'auth.login');
  res.json({ user: publicUser(user), accessToken: signAccessToken(user), refreshToken: signRefreshToken(user) });
}
```

Рис. 2 функция login

Для защиты маршрутов используются middleware проверки авторизации и ролей пользователей см. Рисунок 3.

```
export async function authRequired(req, res, next) {
  try {
    const header = req.headers.authorization || '';
    const token = header.startsWith('Bearer ') ? header.slice(7) : null;
    if (!token) return res.status(401).json({ error: { code: 'NO_TOKEN', message: 'Требуется авторизация' } });
    const payload = jwt.verify(token, process.env.JWT_ACCESS_SECRET);
    const user = await User.findById(payload.sub);
    if (!user || user.is_banned) return res.status(401).json({ error: { code: 'USER_BLOCKED', message: 'Пользователь заблокирован или не найден' } });
    req.user = user;
    next();
  } catch {
    res.status(401).json({ error: { code: 'INVALID_TOKEN', message: 'Недействительный JWT-токен' } });
  }
}

export function optionalSessionCheck(req, res, next) {
  // Сессия используется как дополнительный серверный слой контроля для браузерного клиента.
  if (req.session) req.session.lastSeen = Date.now();
  next();
}
```

Рис. 3 middleware

2.4 Реализация каталога комплектующих

Каталог позволяет пользователю просматривать доступные комплектующие.

Функциональные возможности каталога включают:

- просмотр списка компонентов;
- фильтрацию по категориям;
- поиск по названию;
- сортировку по стоимости и другим параметрам.

Каждый товар содержит:

- наименование;
- технические характеристики;
- цену;
- производителя;
- статус наличия.

Это обеспечивает удобный поиск и подбор необходимых комплектующих.

В приложении Г предоставлен интерфейс фильтров.

2.5 Реализация конфигуратора ПК

Конфигуратор является ключевым модулем системы. Пользователь последовательно выбирает комплектующие различных категорий:

- процессор;
- материнская плата;
- оперативная память;
- видеокарта;
- накопитель;
- блок питания;
- корпус.

В приложении Д предотавлен интерфейс страницы конфигуратора.

						КР-09.02.07-О-19-26ПЗ	Лист
							35
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

После выбора компонентов система:

- проверяет совместимость комплектующих;
- рассчитывает общую стоимость сборки;
- анализирует корректность конфигурации;
- формирует рекомендации по улучшению.

Проверка совместимости позволяет выявлять потенциальные конфликты между компонентами ещё до сохранения сборки.

2.6 Реализация рекомендательной системы

В проекте реализован отдельный сервис рекомендаций.

Он анализирует текущую конфигурацию пользователя и предлагает варианты улучшения. Рекомендации могут касаться:

- повышения производительности;
- оптимизации стоимости;
- улучшения баланса компонентов;
- устранения несовместимости.

Логика формирования рекомендаций вынесена в отдельный сервисный модуль, что упрощает поддержку и дальнейшее развитие функционала.

2.7 Административная часть системы

Административный раздел предназначен для управления содержимым системы.

Администратор может:

- добавлять новые комплектующие;
- редактировать существующие позиции;
- удалять устаревшие товары;
- управлять пользователями;
- просматривать сохранённые конфигурации;
- настраивать рекомендации.

Доступ к административному функционалу ограничен ролью администратора. В приложении Е предоставлен интерфейс административной панели.

2.8 Клиентская часть и пользовательский интерфейс

Пользовательский интерфейс реализован как одностраничное приложение на Vue 3.

Основные преимущества такого подхода:

- высокая скорость работы интерфейса;
- отсутствие полной перезагрузки страниц;
- удобное взаимодействие с API;
- динамическое обновление данных.

Интерфейс включает:

- главную страницу;
- каталог комплектующих;
- страницу конфигуратора;
- страницу авторизации и регистрации;
- личный кабинет;
- раздел сохранённых конфигураций;

2.9 Развёртывание и запуск проекта

Разработанный веб-сервис полностью подготовлен к развёртыванию в контейнеризированной среде с использованием технологии Docker. Применение контейнеризации позволяет упаковать приложение, его зависимости, веб-сервер и базу данных в изолированные контейнеры, что обеспечивает стабильную и предсказуемую работу системы независимо от используемого оборудования или операционной системы. Для оркестрации всех сервисов используется Docker Compose. С его помощью одновременно запускаются несколько взаимосвязанных контейнеров: контейнер с Django-приложением, контейнер с базой данных MySQL, а также дополнительные сервисы, необходимые для корректной работы проекта. Все параметры

						КР-09.02.07-О-19-26ПЗ	Лист
							37
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

запуска, сетевые настройки, тома для хранения данных и зависимости между сервисами описаны в едином конфигурационном файле `docker-compose.yml`. Перед первым запуском проекта необходимо установить Docker и Docker Compose. После установки достаточно открыть терминал в корневой директории проекта и выполнить команду:

```
docker-compose up --build.
```

В приложении Ж предоставлена часть кода Dockerfile.

Данная команда выполняет сборку Docker-образов на основе Dockerfile, загружает необходимые зависимости, создаёт контейнеры и запускает их. При первом запуске этот процесс может занять некоторое время, так как происходит установка всех требуемых библиотек и инициализация базы данных. После успешного запуска контейнеров веб-приложение становится доступным по локальному адресу, указанному в настройках проекта, например `http://localhost:8000`. База данных и остальные сервисы начинают работать автоматически в рамках единой Docker-сети, что исключает необходимость их ручной настройки. После инициализации системы необходимо выполнить миграции базы данных и заполнить её тестовыми данными. Для этого используется команда: `docker-compose exec web python manage.py seed` Seed-команда автоматически создаёт начальный набор данных, необходимый для демонстрации и тестирования работы системы. В базу данных добавляются категории комплектующих, тестовые товары, демонстрационные пользователи и другие связанные записи. Использование Docker даёт проекту ряд важных преимуществ. Прежде всего, обеспечивается единообразие среды выполнения: приложение работает одинаково как на компьютере разработчика, так и на тестовом или рабочем сервере. Это устраняет проблему несовместимости версий библиотек, интерпретатора Python или системы управления базами данных. Кроме того, контейнеризация значительно упрощает процесс развёртывания. Для запуска проекта на новом компьютере или сервере достаточно установить Docker,

						КР-09.02.07-О-19-26ПЗ	Лист
							38
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

скопировать исходный код и выполнить одну команду. Все необходимые зависимости и настройки будут автоматически применены внутри контейнеров. Дополнительным преимуществом является изоляция сервисов. Веб-приложение, база данных и другие компоненты работают независимо друг от друга, что повышает надёжность системы и упрощает сопровождение. При необходимости любой контейнер можно перезапустить, обновить или заменить без влияния на остальные части проекта. Также Docker облегчает перенос проекта между различными серверами и операционными системами. Благодаря стандартизированному окружению приложение может быть развёрнуто практически на любой платформе, поддерживающей контейнеризацию. Это особенно важно при дальнейшем развитии проекта, его публикации в облачной инфраструктуре или переносе на промышленный сервер. Таким образом, использование Docker и Docker Compose делает процесс установки, запуска и сопровождения веб-сервиса быстрым, удобным и надёжным, а также обеспечивает готовность проекта к дальнейшему масштабированию и промышленной эксплуатации.

						КР-09.02.07-О-19-26ПЗ	Лист
							39
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был разработан веб-сервис конфигурирования персональных компьютеров, предназначенный для автоматизации процесса подбора комплектующих с учётом их совместимости, стоимости и требований пользователя. Разработанное программное решение позволяет значительно упростить процесс выбора компонентов, минимизировать вероятность ошибок при сборке и повысить удобство взаимодействия пользователей с системой.

В процессе выполнения работы были решены все поставленные задачи. Проведён анализ предметной области и существующих аналогов, определены основные функциональные и нефункциональные требования к разрабатываемой системе. На основе полученных результатов была спроектирована архитектура приложения, разработана структура базы данных и реализована клиент-серверная модель взаимодействия.

Практическая часть работы включала разработку серверной части приложения с использованием современных технологий веб-разработки, реализацию пользовательского интерфейса, настройку системы аутентификации и авторизации, создание механизмов фильтрации и подбора комплектующих, а также организацию взаимодействия между всеми модулями системы. Особое внимание было уделено обеспечению безопасности пользовательских данных, проверке совместимости компонентов и удобству использования интерфейса.

В результате был создан полнофункциональный веб-сервис, предоставляющий пользователям возможность регистрации, авторизации, подбора комплектующих, формирования собственных сборок и сохранения результатов. Реализованная система обладает модульной архитектурой, что обеспечивает её масштабируемость, расширяемость и возможность дальнейшего развития.

						КР-09.02.07-О-19-26ПЗ	Лист
							40
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Разработанный проект может быть использован как основа для создания коммерческого онлайн-сервиса по подбору компьютерных комплектующих. В дальнейшем функциональность системы может быть расширена за счёт внедрения интеллектуальных алгоритмов рекомендаций, интеграции с внешними торговыми площадками, реализации системы сравнения сборок, а также добавления аналитических и рекомендательных модулей.

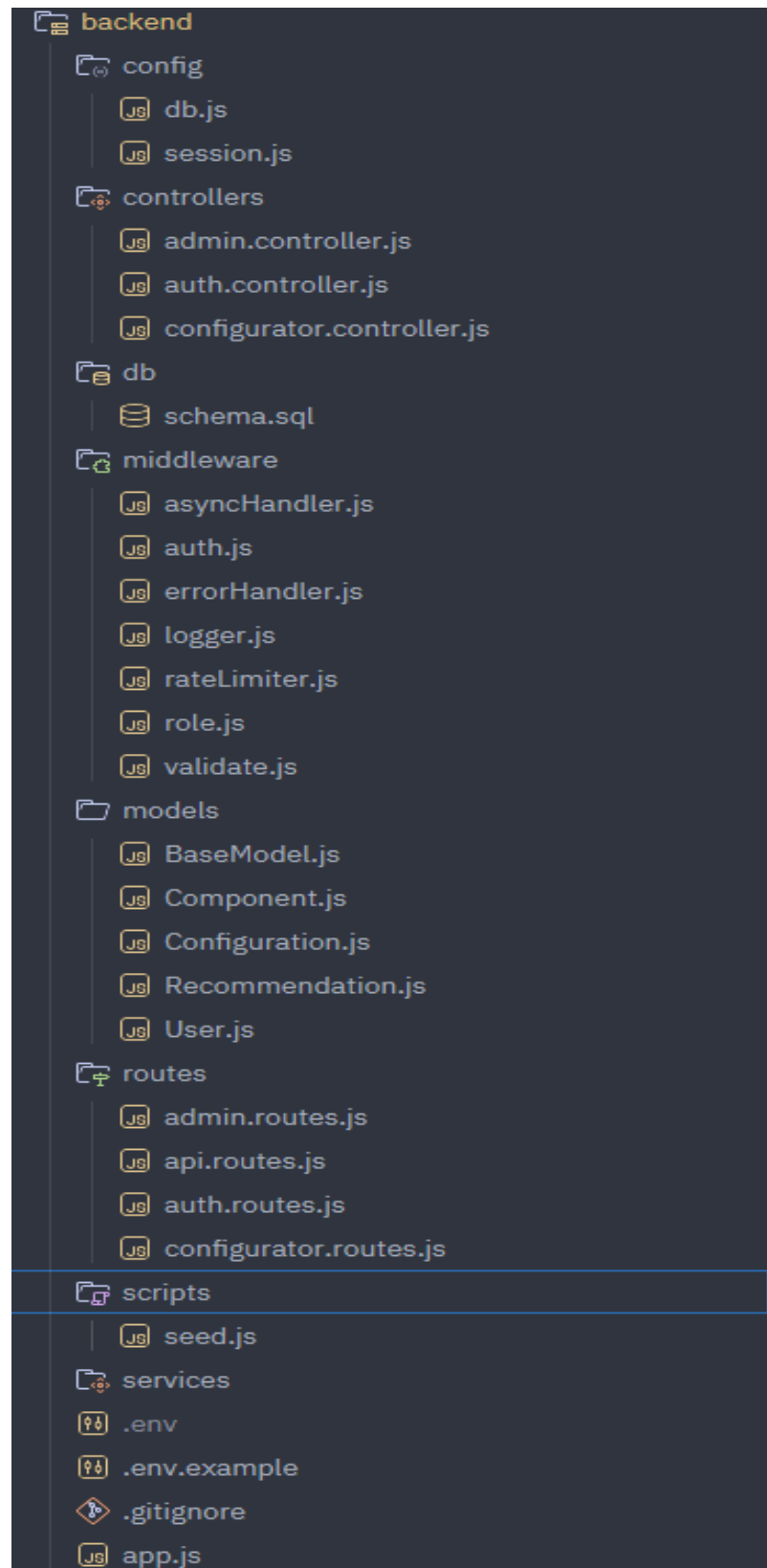
Таким образом, цель курсовой работы была полностью достигнута, а разработанный веб-сервис успешно решает поставленные задачи и демонстрирует практическое применение современных технологий веб-разработки при создании информационных систем.

						КР-09.02.07-О-19-26ПЗ	Лист
							41
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

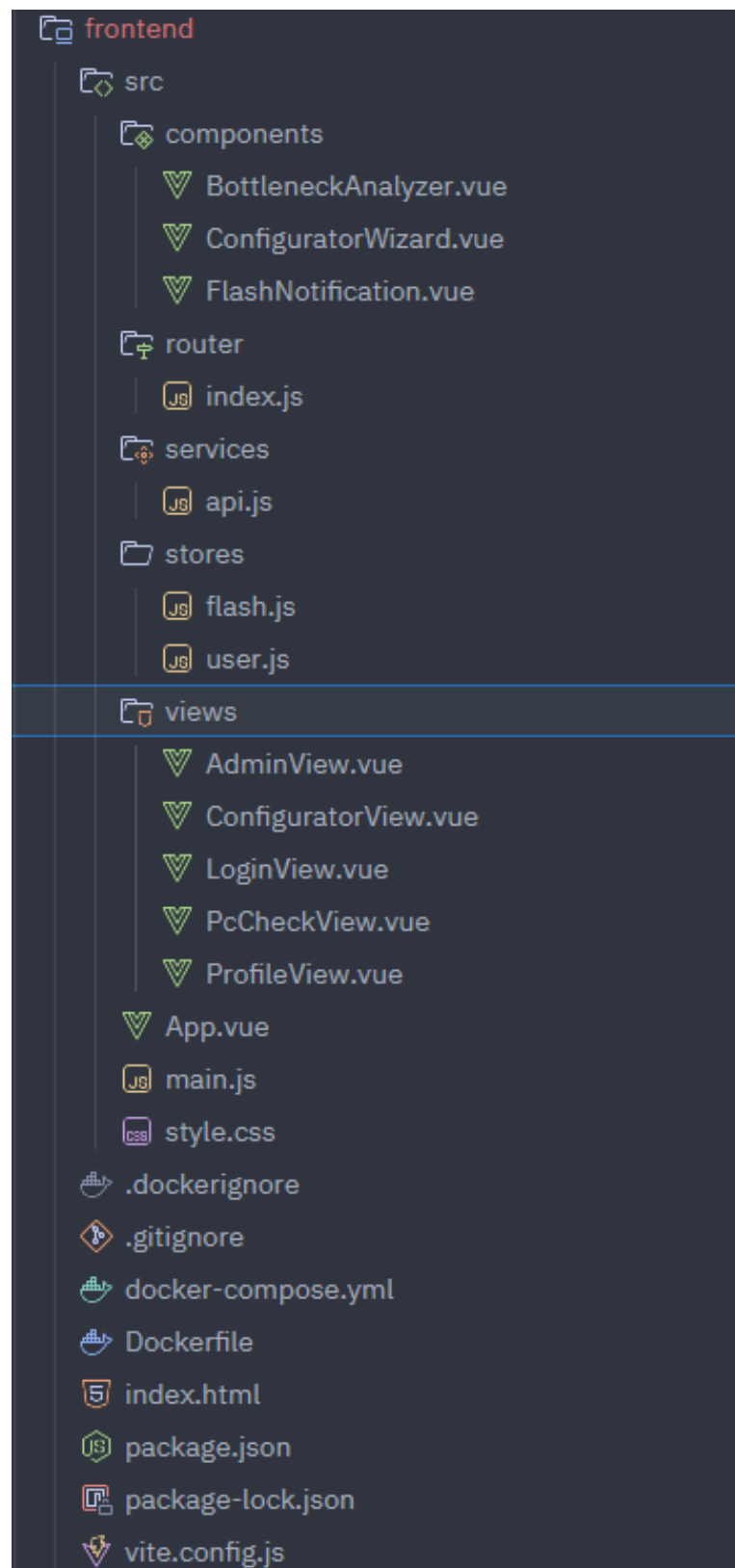
Список используемых источников

1. Флэнаган Д. JavaScript. Подробное руководство. — Санкт-Петербург: Питер, 2023. — 704 с.
2. Фримен Э., Робсон Э. Изучаем программирование на JavaScript. — Санкт-Петербург: Питер, 2022. — 640 с.
3. Banks A., Porcello E. Learning React. — Sebastopol: O'Reilly Media, 2023. — 350 p.
4. Burkov A. Node.js Design Patterns. — Birmingham: Packt Publishing, 2022. — 526 p.
5. Freeman A. Pro ASP.NET Core. — New York: Apress, 2023. — 1040 p.
6. Документация React [Электронный ресурс]. — Режим доступа: <https://react.dev>
7. Документация Node.js [Электронный ресурс]. — Режим доступа: <https://nodejs.org>
8. Документация Express.js [Электронный ресурс]. — Режим доступа: <https://expressjs.com>
9. Документация MongoDB [Электронный ресурс]. — Режим доступа: <https://www.mongodb.com/docs>
10. Документация Docker [Электронный ресурс]. — Режим доступа: <https://docs.docker.com>
11. Документация JWT [Электронный ресурс]. — Режим доступа: <https://jwt.io>
12. Документация Redux Toolkit [Электронный ресурс]. — Режим доступа: <https://redux-toolkit.js.org>
13. Липаев В.В. Программная инженерия сложных программных систем. — Москва: СИНТЕГ, 2021. — 432 с.
14. Соммервилл И. Инженерия программного обеспечения. — Москва: Вильямс, 2021. — 928 с.
15. ГОСТ 19.701–90. Схемы алгоритмов, программ, данных и систем. — Москва: Издательство стандартов, 1991.

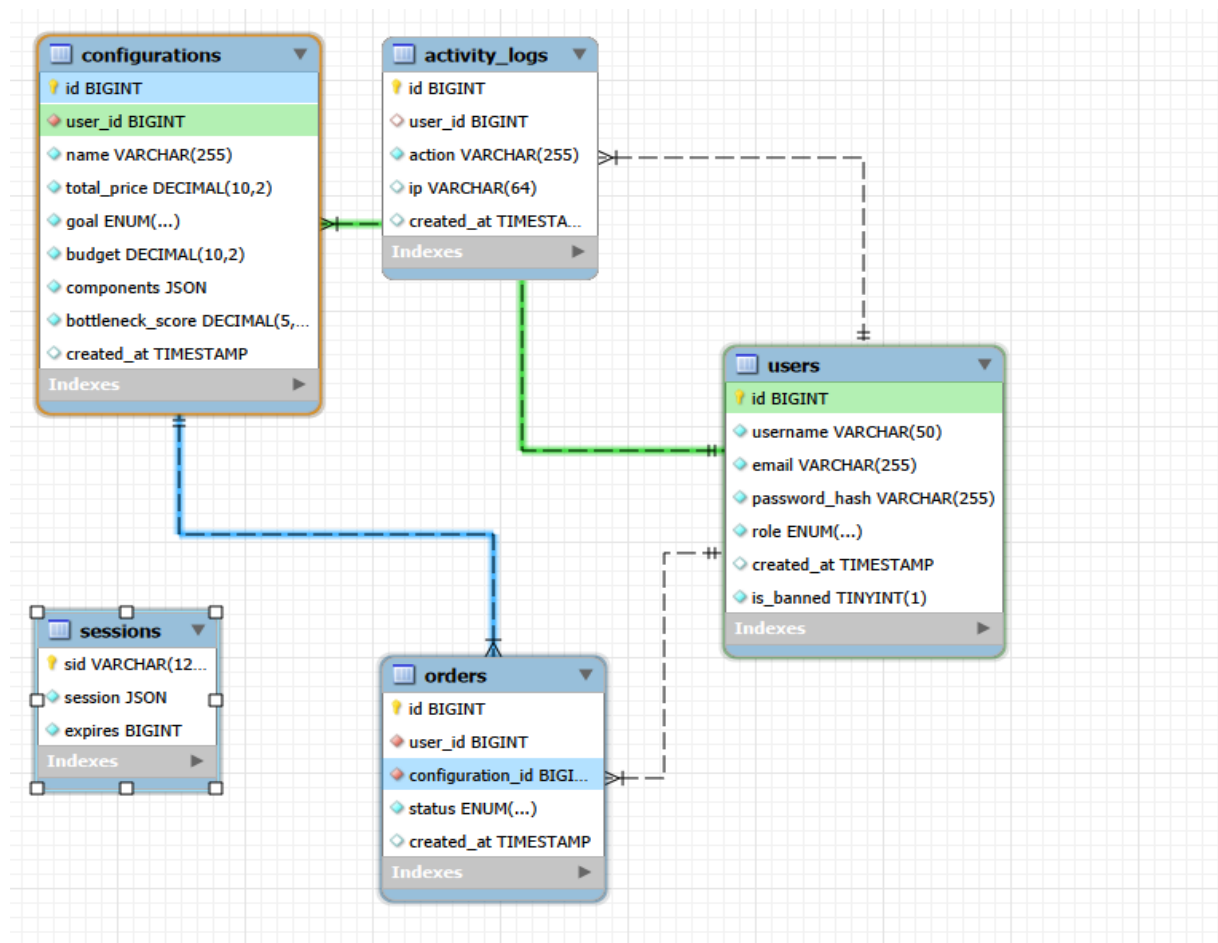
Структура архитектуры backend



Структура архитектуры frontend



Ег-диаграмма базы данных



Интерфейс фильтров

🔍 Поиск по фильтрам

^ Наличие в магазинах

☒ В наличии

☒ Под заказ: сегодня

☒ Под заказ: завтра

☐ Отсутствующие в продаже

☒ Совместимые товары (5)

☐ Рейтинг 4 и выше

☐ Надёжные модели

^ Цена

от

до

^ Бренд

☐ AMD

☐ Intel

Применить

Сбросить

Страница конфигуратора

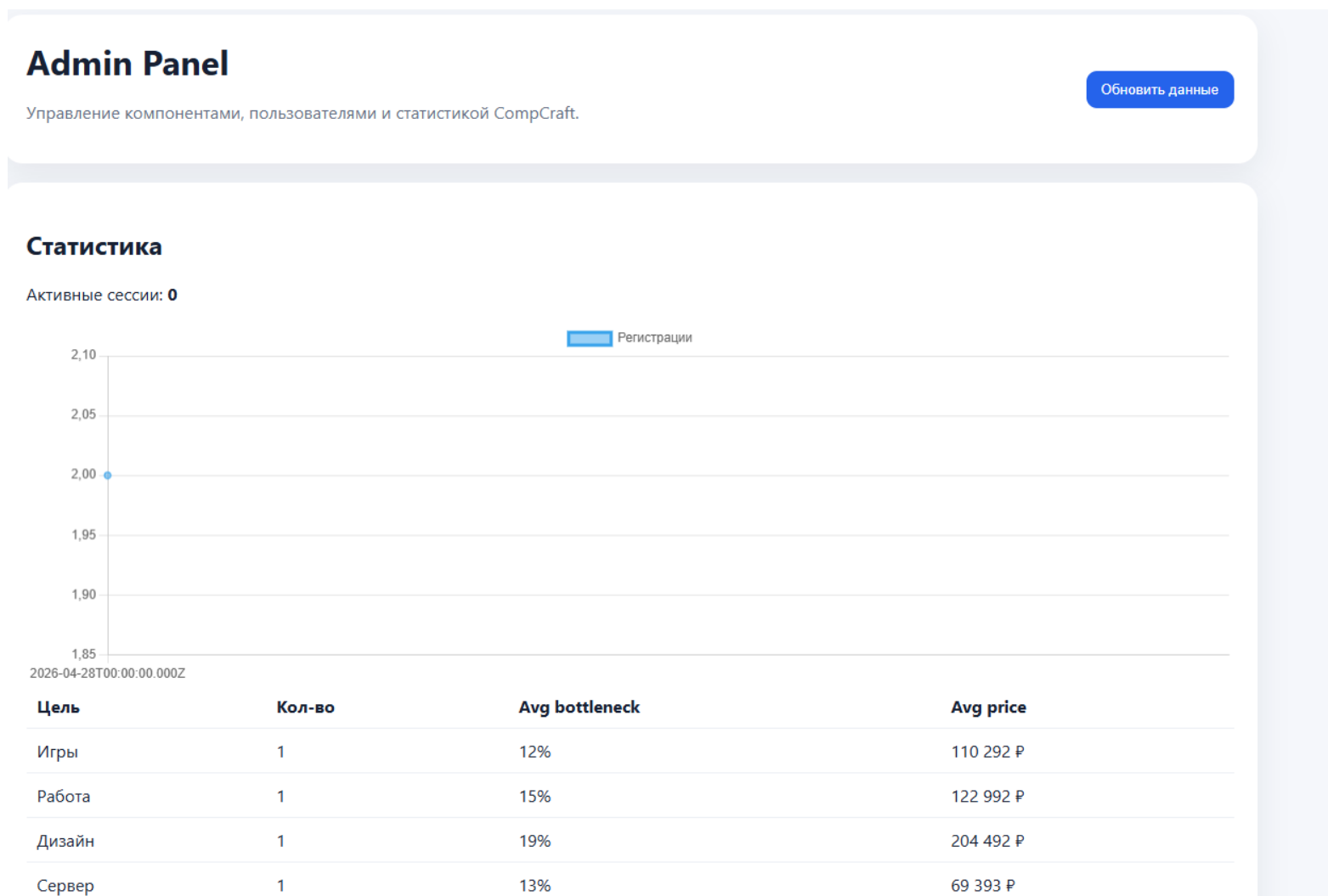
Обязательные комплектующие

Автоподбор

Сохранить 0/7 ?

	Процессор Обязательно	Добавить
	Мат. плата Обязательно	Добавить
	Блок питания Обязательно	Добавить
	Корпус Обязательно	Добавить
	Видеокарта Опционально	Добавить
	Охлаждение процессора Обязательно	Добавить
	Оперативная память Обязательно	Добавить
	Накопители Обязательно	Добавить
	Звуковая карта Опционально	Добавить

Интерфейс админ панели



Часть кода Dockerfile

```
1
2
3  # ----- Frontend build -----
4  FROM node:20-bookworm AS frontend-build
5  WORKDIR /app/frontend
6  COPY frontend/package*.json ./
7  RUN npm install
8  COPY frontend/ ./
9  ARG VITE_API_URL=/api/v1
10 ENV VITE_API_URL=${VITE_API_URL}
11 RUN npm run build
12
13 # ----- Backend dependencies -----
14 FROM node:20-bookworm AS backend-build
15 WORKDIR /app/backend
16 COPY backend/package*.json ./
17 RUN npm install --omit=dev
18 COPY backend/ ./
19
20
21 FROM node:20-bookworm
22
23 ENV DEBIAN_FRONTEND=noninteractive
24 RUN apt-get update \
25     && apt-get install -y --no-install-recommends \
26         mariadb-server \
27         mariadb-client \
28         nginx \
29         tini \
30         ca-certificates \
31     && rm -rf /var/lib/apt/lists/*
32
33 WORKDIR /app
```